FinGuard AI test cases

30 test cases for scoring, access control, authentication and the learning loop. Each case is automated with pytest in src/tests and the test name is in the last column.

Project name	Class / roll no.	App version	Environment	Execution date
FinGuard AI	1024030068 1024030067	1.0 (prototype)	Local machine. Python venv, pytest, FastAPI test client, temporary SQLite database.	2026-11-08

Test case ID	Module	Scenario / objective	Test type	Preconditions	Test data	Test steps	Expected result	Actual result	Status	Priority	Evidence / defect ID
FG-SCORE-001	Scoring	Risk signal weights add up to 100%	Positive	Backend code is loaded. No login needed.	Weights: ML 0.50, Rules 0.20, Behavior 0.15, Device 0.10, Geography 0.05	1. Read the weight table from the aggregator. 2. Add the five weights.	The total is exactly 1.0.	The total was 1.0.	**Pass**	High	tests/test_aggregator.py::test_weights_sum_to_one
FG-SCORE-002	Scoring	Score an ordinary daytime payment as Low risk	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). Signed in as analyst.	Amount: 40 Hour: 14 Category: Grocery Foreign: no Location mismatch: no Device trust: 85 Transactions in 24h: 1 Age: 40	1. Send POST /score with the test data. 2. Read the score and tier in the response.	Score is 30 or below and the tier is Low.	Score was within 0 to 30 and the tier was Low.	**Pass**	High	tests/test_aggregator.py::test_normal_transaction_is_low
FG-SCORE-003	Scoring	Score a risky payment well above a normal one	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). Signed in as analyst.	Normal payment: Amount: 40 Hour: 14 Category: Grocery Foreign: no Location mismatch: no Device trust: 85 Transactions in 24h: 1 Age: 40 Risky payment: Amount: 100000 Hour: 3 Category: Electronics Foreign: yes Location mismatch: yes Device trust: 10 Transactions in 24h: 8 Age: 35	1. Score the normal payment and note the score. 2. Score the risky payment and note the score. 3. Compare the two.	The risky score is at least 45 and more than 30 points above the normal score.	The risky score met both conditions.	**Pass**	High	tests/test_aggregator.py::test_risky_transaction_scores_well_above_normal
FG-SCORE-004	Scoring	Every score comes with an explanation field	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). Signed in as analyst.	Amount: 100000 Hour: 3 Category: Electronics Foreign: yes Location mismatch: yes Device trust: 10 Transactions in 24h: 8 Age: 35	1. Send POST /score with the test data. 2. Check the response for a drivers field.	The response contains a drivers list. It is filled when the models are trained.	The drivers field was present in the response.	**Pass**	Medium	tests/test_aggregator.py::test_response_has_drivers_key
FG-ML-001	ML model	ML signal stays between 0 and 100	Boundary	Models are trained with ml/train.py and the files exist in src/ml/artifacts/.	Amount: 900 Hour: 3 Category: Electronics Foreign: yes Location mismatch: yes Device trust: 20 Transactions in 24h: 6 Age: 35	1. Build the feature vector for the test data. 2. Run the XGBoost and Isolation Forest scoring. 3. Read the combined ML signal.	The ML signal is between 0 and 100 inclusive.	The ML signal was inside the range.	**Pass**	High	tests/test_ml.py::test_ml_score_is_in_range
FG-ML-002	ML model	Explanation covers every feature	Positive	Models are trained with ml/train.py.	Amount: 900 Hour: 3 Category: Electronics Foreign: yes Location mismatch: yes Device trust: 20 Transactions in 24h: 6 Age: 35	1. Request the model drivers (TreeSHAP) for the test data. 2. List the feature names returned. 3. Add up the absolute shares.	Eight drivers are returned: Amount, Hour of day, Foreign transaction, Location mismatch, Device trust, Velocity (24h), Cardholder age and Merchant category. The absolute shares total roughly 100% (90 to 110 after rounding).	All eight drivers were present and the shares were in range.	**Pass**	Medium	tests/test_ml.py::test_drivers_cover_every_feature

Test case ID	Module	Scenario / objective	Test type	Preconditions	Test data	Test steps	Expected result	Actual result	Status	Priority	Evidence / defect ID
FG-ACCESS-001	Access control	Scoring requires a login	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Request: POST /score Authorization header: none Body: a normal payment	1. Send POST /score with no Authorization header.	The API answers 401.	The API answered 401.	**Pass**	High	tests/test_api.py::test_score_needs_login
FG-ACCESS-002	Access control	Staff can score, a customer cannot	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Analyst and customer accounts Body: a normal payment	1. Sign in as analyst and send POST /score. 2. Sign in as customer and send POST /score.	The analyst gets 200. The customer gets 403.	The analyst got 200 and the customer got 403.	**Pass**	High	tests/test_api.py::test_staff_can_score_customer_cannot
FG-ACCESS-003	Access control	A customer cannot list the case queue	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Customer account r.mehta	1. Sign in as customer. 2. Send GET /cases.	The API answers 403.	The API answered 403.	**Pass**	High	tests/test_api.py::test_customer_cannot_see_cases
FG-ACCESS-004	Access control	Only the analyst can record a decision	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). Case FG-10432 is open.	Case: FG-10432 Decision: Confirmed fraud	1. Sign in as admin and send POST /decision. 2. Sign in as analyst and send the same request. 3. As analyst, send the same request again.	Admin gets 403. The analyst's first request gets 200. The repeat gets 409 because the case is already decided.	The responses were 403, 200 and 409.	**Pass**	High	tests/test_api.py::test_only_analyst_can_decide
FG-ACCESS-005	Access control	A decision appears in the audit log with the deciding role	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). Case FG-10431 is open.	Case: FG-10431 Decision: False positive	1. Sign in as analyst and record the decision. 2. Sign in as admin. 3. Send GET /audit and read the newest entry.	The newest entry shows case FG-10431 and the role analyst.	The newest entry matched.	**Pass**	Medium	tests/test_api.py::test_decision_lands_in_audit_log
FG-ACCESS-006	Access control	A customer sees only their own cases	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Customer account r.mehta (name R. Mehta) Analyst account	1. Sign in as customer and send GET /my-cases. 2. Check the owner of every returned case. 3. Sign in as analyst and send GET /my-cases.	The customer gets 200 and every case belongs to R. Mehta. The analyst gets 403.	Only R. Mehta's cases were returned, and the analyst got 403.	**Pass**	High	tests/test_api.py::test_customer_sees_only_own_cases
FG-ACCESS-007	Access control	Form metadata is open to any signed-in user	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Analyst account Customer account No account	1. Send GET /meta as analyst. 2. Send GET /meta as customer. 3. Send GET /meta with no token.	The analyst gets 200 with a non-empty category list. The customer gets 200. No token gets 401.	The responses were 200, 200 and 401.	**Pass**	Low	tests/test_api.py::test_meta_lists_categories_for_any_signed_in_user
FG-AUTH-001	Authentication	Sign in with correct credentials	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Username: analyst Password: finguard-demo	1. Send POST /auth/login with the test data. 2. Read the response.	The API answers 200 with a token and the role analyst.	A token and the role analyst were returned.	**Pass**	High	tests/test_auth.py::test_login_returns_token_and_role
FG-AUTH-002	Authentication	Reject a wrong password and an unknown user	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Attempt 1: admin / nope Attempt 2: ghost / finguard-demo	1. Send POST /auth/login with attempt 1. 2. Send POST /auth/login with attempt 2.	Both attempts get 401.	Both attempts got 401.	**Pass**	High	tests/test_auth.py::test_wrong_password_and_unknown_user_are_rejected
FG-AUTH-003	Authentication	The current-user endpoint returns the signed-in role	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Risk manager account	1. Sign in as risk. 2. Send GET /auth/me with the token.	The response shows the role risk.	The role risk was returned.	**Pass**	Medium	tests/test_auth.py::test_me_returns_identity
FG-AUTH-004	Authentication	A role header grants nothing	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Header: x-role: admin Token: none	1. Send GET /cases with the x-role header and no token.	The API answers 401. The header does not grant access.	The API answered 401.	**Pass**	High	tests/test_auth.py::test_a_role_header_no_longer_grants_anything

Test case ID	Module	Scenario / objective	Test type	Preconditions	Test data	Test steps	Expected result	Actual result	Status	Priority	Evidence / defect ID
FG-AUTH-005	Authentication	Reject a token that has been altered	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	A valid admin token with one extra character added	1. Sign in as admin and copy the token. 2. Add one character to the end. 3. Send GET /auth/me with the altered token.	The API answers 401.	The API answered 401.	**Pass**	High	tests/test_auth.py::test_tampered_token_is_rejected
FG-AUTH-006	Authentication	Reject an expired token	Boundary	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	A token for admin whose expiry time was 10 seconds ago	1. Create a token with an expiry in the past, signed with the server key. 2. Send GET /auth/me with it.	The API answers 401.	The API answered 401.	**Pass**	High	tests/test_auth.py::test_expired_token_is_rejected
FG-AUTH-007	Authentication	Reject a token signed with a different key	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	A token for admin signed with the key 'not-the-secret'	1. Create the forged token. 2. Send GET /auth/me with it.	The API answers 401.	The API answered 401.	**Pass**	High	tests/test_auth.py::test_token_signed_with_another_key_is_rejected
FG-AUTH-008	Authentication	Login is blocked after five failed attempts	Boundary	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). No earlier failed attempts for admin.	Username: admin Wrong password: bad Correct password: finguard-demo	1. Send five logins with the wrong password. 2. Send a sixth login with the wrong password. 3. Send a login with the correct password.	The first five get 401. The sixth gets 429. The correct password also gets 429 until the one-minute window passes.	The responses were five 401s, then 429 and 429.	**Pass**	High	tests/test_auth.py::test_login_is_rate_limited
FG-AUTH-009	Authentication	Passwords are stored as salted hashes	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Account: admin	1. Read the admin row from the users table. 2. Compare the stored value with the password.	The stored value is a 64-character hash with a salt, and it is not the password.	The stored value was a 64-character hash with a salt.	**Pass**	High	tests/test_auth.py::test_passwords_are_stored_hashed
FG-LEARN-001	Learning loop	A high-risk payment opens a case for the analyst	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). Customer is signed in.	Amount: 100000 Hour: 3 Category: Electronics Foreign: yes Location mismatch: yes Device trust: 10 Transactions in 24h: 8 Age: 35 (The automated test forces the Critical tier so this path always runs.)	1. Send POST /transactions as customer with the test data. 2. Read the status and case reference. 3. Sign in as analyst and send GET /cases.	The response status is blocked and includes a case reference. The case is in the analyst's queue.	The payment was blocked and the case appeared in the queue.	**Pass**	High	tests/test_learning.py::test_risky_payment_opens_a_case
FG-LEARN-002	Learning loop	A low-risk payment opens no case	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). Customer is signed in.	Amount: 40 Hour: 14 Category: Grocery Foreign: no Location mismatch: no Device trust: 85 Transactions in 24h: 1 Age: 40 (The automated test forces the Low tier.)	1. Send POST /transactions as customer with the test data. 2. Read the status and case reference.	The status is approved and there is no case reference.	The payment was approved with no case.	**Pass**	High	tests/test_learning.py::test_low_risk_payment_opens_no_case
FG-LEARN-003	Learning loop	Confirming fraud labels the stored payment as 1	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). A blocked payment has a case.	Decision: Confirmed fraud	1. Make a high-risk payment as customer. 2. As analyst, record Confirmed fraud on its case. 3. Read the labelled records from the database.	One labelled record exists, its label is 1 and it holds the payment's amount.	One record was labelled 1 with the right amount.	**Pass**	High	tests/test_learning.py::test_analyst_decision_labels_the_stored_record
FG-LEARN-004	Learning loop	Marking a false positive labels the stored payment as 0	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). A blocked payment has a case.	Decision: False positive	1. Make a high-risk payment as customer. 2. As analyst, record False positive on its case. 3. Read the labelled records from the database.	One labelled record exists and its label is 0.	One record was labelled 0.	**Pass**	High	tests/test_learning.py::test_analyst_decision_labels_the_stored_record

Test case ID	Module	Scenario / objective	Test type	Preconditions	Test data	Test steps	Expected result	Actual result	Status	Priority	Evidence / defect ID
FG-LEARN-005	Learning loop	Escalating leaves the payment unlabelled	Boundary	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). A blocked payment has a case.	Decision: Escalated	1. Make a high-risk payment as customer. 2. As analyst, record Escalated on its case. 3. Read the labelled records and the model counts.	No labelled records exist and the model counts show one unlabelled record.	There were no labels and one unlabelled record.	**Pass**	Medium	tests/test_learning.py::test_escalation_leaves_the_record_unlabelled
FG-LEARN-006	Learning loop	Role limits on payments, retraining and the model page	Negative	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo).	Analyst account Customer account	1. As analyst, send POST /transactions. 2. As analyst, send POST /retrain. 3. As customer, send GET /model.	Each request gets 403.	Each request got 403.	**Pass**	High	tests/test_learning.py::test_role_limits
FG-LEARN-007	Learning loop	Retraining is refused without enough labelled records	Boundary	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). No labelled records exist.	Records labelled: 0 (minimum is 20, with 5 confirmed frauds)	1. Sign in as admin and send POST /retrain. 2. Read the response. 3. Send GET /model and read the version history.	The request returns 200 with promoted false and a reason. The history shows one entry.	Promoted was false and the history had one entry.	**Pass**	High	tests/test_learning.py::test_retrain_refuses_without_enough_labels
FG-LEARN-008	Learning loop	Retraining on new labelled records returns a champion/challenger result	Positive	Backend is running on a fresh database with the four demo accounts (customer r.mehta, analyst, risk, admin; password finguard-demo). Models have been trained with ml/train.py. The dataset is in src/ml/data/.	Simulated records: 60 labelled (30 fraud, 30 legitimate)	1. Sign in as admin. 2. Send POST /demo/new-pattern. 3. Send POST /retrain. 4. Read the response.	The response contains champion_pr_auc, challenger_pr_auc and promoted. The live model is replaced only if both checks pass.	All three fields were present.	**Pass**	Medium	tests/test_learning.py::test_retrain_end_to_end